

Room - Tipos de Dados e Anotações

Tipos de Dados:

a) Tipos Primitivos e Wrappers Suportados (Nativamente)

- **Texto:** String, Char
- **Números Inteiros:** Int, Long, Short, Byte
- **Números Decimais:** Float, Double
- **Lógico:** Boolean (armazenado no banco como 0 ou 1)
- **Binário:** ByteArray (ideal para salvar imagens, arquivos, etc.)

b) Tipos Complexos (Requerem @TypeConverter)

Por padrão, o Room não sabe como salvar classes customizadas ou listas nativas.

Você precisa criar uma classe para converter esses tipos para String ou ByteArray:

- Listas e Coleções: List<String>, Set<Int>, etc.
- Datas: java.util.Date, java.time.LocalDate
- Objetos Customizados: Usuario, Endereco, etc.
- Enums: Enum (pode ser convertido para String ou Int)

Anotações:

@Entity: Define que a classe Kotlin representa uma tabela no banco de dados SQLite.

@PrimaryKey: Identifica o campo que serve como a chave primária única para cada registro da tabela.

@ColumnInfo: Modifica as propriedades da coluna, permitindo alterar o nome no banco e configurar outras características.

@Ignore: Indica que o Room deve ignorar o campo, não criando uma coluna para ele no banco de dados.

@Embedded: Inclui os campos de uma classe externa diretamente como colunas na tabela da Entidade atual.

@ForeignKey: Define uma restrição de integridade referencial para vincular e sincronizar registros entre duas tabelas diferentes.

@Index: Cria índices em colunas específicas para acelerar a velocidade de busca e garantir a unicidade de valores.

@TypeConverters: Especifica quais classes conversoras o Room deve usar para salvar tipos de dados complexos nessa entidade.

@Relation: Define relacionamentos de um-para-muitos ou muitos-para-muitos dentro de classes POJO auxiliares de consulta.

@Junction: Declara a tabela associativa intermediária que une duas entidades em um relacionamento de muitos-para-muitos.

@NonNull: Força uma coluna Java ou Kotlin a rejeitar valores nulos diretamente no esquema do banco de dados SQLite.

Exemplo:

```
import androidx.room.*
```

```
// =====
```

```
// ENTIDADE PRODUTO (Tabela Única)
```

```
// =====
```

// 1. **@Entity**: Define que a classe Kotlin representa uma tabela no banco de dados SQLite.

```
@Entity(
```

```
    tableName = "produtos",
```

// 2. **@Index**: Cria índices em colunas específicas para acelerar buscas e garantir que valores sejam únicos.

```
    indices = [Index(value = ["codigo_barras"], unique = true)]
```

```
)
```

```
data class ProdutoEntity(
```

// 3. **@PrimaryKey**: Identifica o campo que serve como a chave primária única para cada registro.

```
    @PrimaryKey(autoGenerate = true)
```

```
    val id: Long = 0,
```

// 4. **@ColumnInfo**: Modifica propriedades da coluna, como o nome no banco e regras de colação (NOCASE).

```
    @ColumnInfo(name = "codigo_barras")
```

```
    val codigoBarras: String,
```

// 5. **@NonNull**: Força uma coluna a rejeitar valores nulos diretamente no esquema do banco SQLite.

```
    @NonNull
```

```
val nome: String,
```

```
val preco: Double,
```

// 6. @TypeConverters: Especifica a classe conversora que o Room usará para salvar um tipo complexo (Date).

```
@TypeConverters(DataConverter::class)
```

```
val dataCadastro: java.util.Date,
```

// 7. @Embedded: Inclui os campos de uma classe externa diretamente como colunas nesta mesma tabela.

```
@Embedded
```

```
val dimensoes: DimensoesProduto,
```

// 8. @Ignore: Indica que o Room deve ignorar o campo, não criando uma coluna para ele no banco.

```
@Ignore
```

```
val estaMarcadoNoCarrinho: Boolean = false
```

```
)
```

// Classe auxiliar usada pelo @Embedded (Seus campos 'largura' e 'altura' viram colunas na tabela 'produtos')

```
data class DimensoesProduto(
```

```
    val largura: Double,
```

```
    val altura: Double
```

```
)
```

```
// =====
```

```
// CLASSE CONVERSORA (Necessária para o @TypeConverters)
```

```
// =====
```

```
class DataConverter {
```

@TypeConverter

fun deTimestamp(value: Long?): java.util.Date? = value?.let { java.util.Date(it) }

@TypeConverter

fun paraTimestamp(date: java.util.Date?): Long? = date?.time

}